

SQL Complete Flow Summary & Full Practice Database

1. Commands Summary

- **CREATE DATABASE / SCHEMA:** CREATE SCHEMA education;
- **USE DATABASE:** USE education;
- **CREATE TABLE:** CREATE TABLE ...
- **ALTER TABLE:** ADD COLUMN, CHANGE/Rename column, MODIFY data type
- **INSERT:** Add records using INSERT INTO
- **SELECT:** Read records using SELECT
- **UPDATE:** Modify existing records using UPDATE ... WHERE
- **DELETE:** Remove selected records
- **TRUNCATE:** Remove all rows from a table
- **DROP:** Remove the table itself
- **ORDER BY:** Sort query results
- **Aggregate functions:** SUM(), AVG(), COUNT()
- **GROUP BY:** Create groups for aggregate calculations
- **HAVING:** Filter grouped/aggregate results
- **SQL Safe Update:** SET SQL_SAFE_UPDATES = 0/1 when needed

The PDF's final teaching order is: CREATE DATABASE → USE DATABASE → CREATE TABLE → ALTER TABLE → INSERT → SELECT → UPDATE → DELETE/TRUNCATE → DROP → ORDER BY → AGGREGATE FUNCTIONS → GROUP BY → HAVING.

2. Full Integrated Database – education

Run the following in MySQL Workbench or the MySQL Command Prompt. This version keeps the PDF's teacher, student, and employee concepts together so the queries can be practiced on one database.

A. Create and select database

```
CREATE SCHEMA education;  
USE education;
```

B. Create teacher table

```
CREATE TABLE teacher (  
    name VARCHAR(10),
```

```
    age INT,  
    department VARCHAR(20)  
);
```

C. Apply ALTER TABLE commands from the PDF

```
ALTER TABLE teacher  
ADD COLUMN degree VARCHAR(100);
```

```
ALTER TABLE teacher  
CHANGE department dept VARCHAR(20);
```

```
ALTER TABLE teacher  
MODIFY age INT;
```

D. Insert teacher data

```
INSERT INTO teacher (name, age, dept, degree) VALUES  
( 'sri', 24, 'cse', 'B.E'),  
( 'anand', 32, 'it', 'M.E'),  
( 'meena', 28, 'ece', 'M.E'),  
( 'kavin', 30, 'cse', 'Ph.D');
```

E. View teacher data

```
SELECT * FROM teacher;
```

Expected output

name	age	dept	degree
sri	24	cse	B.E
anand	32	it	M.E
meena	28	ece	M.E
kavin	30	cse	Ph.D

F. Update teacher data

```
UPDATE teacher  
SET age = 23  
WHERE name = 'sri';
```

```
SELECT * FROM teacher;
```

Expected output

name	age	dept	degree
sri	23	cse	B.E
anand	32	it	M.E
meena	28	ece	M.E
kavin	30	cse	Ph.D

G. Create student table

```
CREATE TABLE student (  
    stud_name VARCHAR(30),  
    mark INT,  
    department VARCHAR(20)  
);
```

H. Insert student data

```
INSERT INTO student (stud_name, mark, department) VALUES  
(Arun', 85, 'cse'),  
(Bala', 72, 'cse'),  
(Divya', 91, 'it'),  
(Gokul', 65, 'it'),  
(Harini', 88, 'ece'),  
(Karthik', 76, 'ece');
```

I. Basic SELECT queries

```
SELECT *  
FROM student  
WHERE department = 'cse';
```

```
SELECT stud_name, mark  
FROM student;
```

Expected output – CSE students

stud_name	mark	department
Arun	85	cse
Bala	72	cse

J. ORDER BY

```
SELECT stud_name, mark  
FROM student  
ORDER BY mark DESC;
```

Expected output

stud_name	mark
Divya	91
Harini	88
Arun	85
Karthik	76
Bala	72
Gokul	65

K. Aggregate functions

```
SELECT SUM(mark)
FROM student;
```

```
SELECT AVG(mark)
FROM student;
```

Expected output

```
SUM(mark) = 477
AVG(mark) = 79.5000
```

L. GROUP BY

```
SELECT department, SUM(mark)
FROM student
GROUP BY department;
```

```
SELECT department, COUNT(stud_name)
FROM student
GROUP BY department;
```

```
SELECT department, COUNT(stud_name) AS total_students
FROM student
GROUP BY department
ORDER BY total_students;
```

Expected output – GROUP BY

```
department SUM(mark)
cse         157
ece         164
it          156
```

```
department COUNT(stud_name)
cse         2
ece         2
it          2
```

```
department total_students
cse         2
ece         2
it          2
```

M. Create employee table

```
CREATE TABLE employee (
    name VARCHAR(30),
    department VARCHAR(20),
```

```
    salary DECIMAL(10,2)
);
```

N. Insert employee data

```
INSERT INTO employee (name, department, salary) VALUES
('Ravi', 'IT', 55000),
('Priya', 'IT', 65000),
('Suresh', 'HR', 50000),
('Anitha', 'HR', 58000),
('Manoj', 'CSE', 72000),
('Keerthi', 'CSE', 68000);
```

O. WHERE with COUNT

```
SELECT COUNT(*)
FROM employee
WHERE department = 'IT';
```

Expected output

```
COUNT(*) = 2
```

P. GROUP BY department

```
SELECT department, COUNT(*)
FROM employee
GROUP BY department;
```

```
SELECT department, AVG(salary)
FROM employee
GROUP BY department;
```

Expected output

```
department  COUNT(*)
CSE         2
HR          2
IT          2
```

```
department  AVG(salary)
CSE         70000.00
HR          54000.00
IT          60000.00
```

Q. HAVING

```
SELECT department, COUNT(*)
FROM employee
GROUP BY department
HAVING AVG(salary) < 60000;
```

Expected output

```
department  COUNT(*)
HR           2
```

R. DELETE example

```
DELETE FROM teacher
WHERE dept = 'cse';
```

```
SELECT * FROM teacher;
```

Expected output

```
name age dept degree
anand 32  it   M.E
meena 28  ece  M.E
```

S. TRUNCATE example

```
TRUNCATE TABLE teacher;
```

```
SELECT * FROM teacher;
```

Expected output

Empty result set – all rows in teacher are removed, but the table remains.

T. DROP example

```
DROP TABLE teacher;
```

Expected output

Table teacher is removed. Running SELECT * FROM teacher; afterward will produce a table-not-found error.

U. Safe update mode

```
SET SQL_SAFE_UPDATES = 0;
```

```
-- perform the required UPDATE/DELETE
```

```
SET SQL_SAFE_UPDATES = 1;
```

3. Important Command Notes

- `CHANGE department dept VARCHAR(20)` renames the department column to dept and specifies its data type.
- `MODIFY age INT` changes/specifies the data type of the age column.
- `DELETE` removes selected rows when a `WHERE` condition is supplied.
- `TRUNCATE` removes all rows from the table but keeps the table structure.
- `DROP TABLE` removes the table itself.
- `WHERE` filters individual rows before grouping.
- `GROUP BY` groups rows so aggregate functions such as `SUM()`, `AVG()`, and `COUNT()` can be calculated per group.
- `HAVING` filters groups based on aggregate conditions.
- The PDF shows `SQL_SAFE_UPDATES` being switched off and then back on when needed.

4. One-Page Revision Flow

`CREATE SCHEMA` → `USE` → `CREATE TABLE` → `ALTER TABLE` → `INSERT` → `SELECT` → `UPDATE` → `DELETE/TRUNCATE` → `DROP` → `ORDER BY` → `SUM/AVG/COUNT` → `GROUP BY` → `HAVING` → `SAFE UPDATE`